

stable-fips203-mlkem-kem-encaps-k4

FIPS 203 Alg.20 ML-KEM-1024 KEM Encaps 实现方案 (k=4)

ascendc 工程 (定型交付)

2026-07-15

摘要

本文档描述 定型交付算子 `examples/stable/stable-fips203-mlkem-kem-encaps-k4/` 在 Atlas A2 / Ascend910B4 上以 AscendC 实现 FIPS 203 Algorithm 20 ML-KEM.Encaps() (参数集 `ml_kem_1024`, $K = 4$), 经 Alg.17 Encaps_internal 调用 Alg.14 K-PKE.Encrypt, 并在设备侧完成 $H(ek)$ 与 $G(m\|H(ek))$ 后写出密文 c 与共享秘密 K 。

晋级 (2026-07-15): 自 `examples/incubating/exp-fips203-mlkem-kem-encaps-k4/` 复制晋级 (# 验收 #); 预研副本保留。登记表: `docs/specs/fips203-mlkem1024-kem-encaps-baseline-registry.md`。

行为对照 (只读; 禁止把探针当编译依赖): `ascendc-tests/pass-fix-f203-alg20-kem-encaps-device-k4/` (CPU+SIM PASS; c/K max=0; SIM Total tick **721010**; liboqs 分项 KAT CPU×10+SIM×3 PASS)。Encrypt 主体行为对齐 `examples/stable/stable-fips203-mlkem-pke-encrypt-k4/` (Alg.14; SIM 2 launch ≈627k; CPU 5 launch 分叉)。写法对齐 `stable-fips203-mlkem-kem-keygen-k4-实现方案-customspec.tex`、`stable-fips203-mlkem-pke-encrypt-k4-实现方案-customspec.tex`。

生产 I/O (强制; Alg.17 工程形态):

- **输入:** `input/ek_kem.bin` (1568 B, = ek_{PKE}) + `input/m.bin` (32 B, **GM 输入**) + 静态 NTT/INTT LUT (与消息无关)。
- **输出:** 仅 `c.bin` (1568 B) 与 `K.bin` (32 B)。
- $h = H(ek)$ 、 $r \in \mathbb{B}^{32}$ (G 后半; Alg.14 随机性输入) **仅** device UB/workspace; **禁止** Host 预填 r ; **禁止** 将 $h/r/y/e_1/e_2/\hat{A}$ 等中间态作为交付契约落盘。

设备 Launch (强制): 与 stable Encrypt / device Encaps 同拓扑——SIM 恰好 2 次 (`f203_kem_enc_prep` → `f203_encrypt_118_119`); CPU tikicpu 5 次 (第一 launch 换为带 KEM 头的 prep; 其后同 stable)。**禁止** 为 H/G 单独增加第 3 次 launch / 额外 `aclrtSynchronizeStream`。

自包含原则 (stable 强制; 与 device 探针 CMake 策略不同): AscendC / Host 源码 全部 vendored 于本目录; 唯一允许的外部代码依赖: `library/shared/`。**禁止** 编译期 `#include` `ascendc-tests/`、其它 `examples/`、`frozen/`; 写码阶段从 stable Encrypt 一次性复制后切断外链 (I/O / launch 与 device 探针一致; 探针 `STABLE_ENCRYPT_ROOT` 仅为测试便利)。

目录

1 工程约束 (自包含)	2
1.1 允许的外部依赖	2

目录	2
1.2 禁止的外部依赖与不安全路径	2
1.3 相对 device 探针的落点差异	2
1.4 唯一交付路径（默认即全量）	2
1.5 目录布局（计划；写码阶段落地）	3
2 数学语义 (Alg.20 → Alg.17 → Alg.14)	4
2.1 参数集	4
2.2 Algorithm 20（对外业务）与 Alg.17（设备 I/O）	4
2.3 相对 PKE Encrypt 的增量	4
2.4 liboqs 对拍口径	4
2.5 Alg.14 主体（设备，与 Encrypt 同）	5
3 编排与 Launch	5
3.1 SIM（生产路径）	5
3.2 CPU（tikicpu 分叉）	5
3.3 相对 correctness（vendor G5）	5
4 AI Core 规模	5
5 GM 布局与头段同步	6
5.1 KemEncInitHead（仅 block0）	6
5.2 双 AIV 与头段次序（强制）	6
6 踩坑与强制条款（换 Agent 必读）	7
7 全量 API 清单（查阅 CANN 9.0.0）	7
7.1 设备哈希（非 CANN 矢量 API；工程共享库）	8
7.2 评估过但未采用	8
8 数学步骤 → API 覆盖率	9
9 验证	9
9.1 Golden 角色	9
9.2 生产验收（默认；写码完成后）	9
9.3 Gate 建议（写码阶段，非本规格交付物）	10
10 性能（SIM 910B4 参考）	10
11 写码衔接（本规格确认后）	10

1 工程约束（自包含）

1.1 允许的外部依赖

路径	用途
library/shared/shake_x of_kernel/	Prep SampleNTT / PRF (ProcessInline)
library/shared/keccak_ f1600_kernel/	Keccak-f[1600]: Encaps 头 Sha3OneShot (SHA3-256: $H(ek)$; SHA3-512: $G(m h)$)
library/shared/f203_un ified_round/	Encrypt 尾 Compress 统一整数向量（若随 Encrypt vendor）
library/shared/	公共头（shake_general.h、 fips203_device_sha3.hpp）
CANN / tikicpulib	工具链与仿真（非业务源码）

1.2 禁止的外部依赖与不安全路径

- 运行时 / 编译期依赖 ascendc-tests/（含 pass-fix-f203-alg20-kem-encaps-device-k4 /、correctness）、frozen/、其它 examples/*/（一次性 vendor 除外）。
- 子进程调用 stable / 探针 run.sh 再拼 Encaps。
- Host tiny_sha3 / liboqs 参与生产路径的 H/G （KAT / golden 脚本除外）；**禁止** Host 算 $H/G/r/K$ 冒充设备。
- 从 ascendc-tests/frozen/ 或 correctness vendor/（G5）**抄实现** / vendor_sync。
- 为 KEM 头增加独立 launch；Host 预填 r ；将 h/r 落盘为交付物。

1.3 相对 device 探针的落点差异

项	device 探针	本 exp（锁定）
Encrypt 源码	CMake STABLE_ENCRYPT_ROOT 编 译期引用	本目录 prep/+compute/ vendored
KEM 头	本目录 kem/ 并入 prep 前 段	同语义；头实现 vendored 于 kem/
Launch / I/O / GM	SIM 2 / CPU 5； $c+K$	同契约
SIM tick 参考	721010	写码目标同量级

1.4 唯一交付路径（默认即全量）

组件	锁定配置
KEM 头	Launch-1 前段: <code>KemEncInitHead</code> （仅 block0）: $m_{\text{GM}} \rightarrow H(\text{ek}) \rightarrow G \rightarrow K \ r$
Prep $\hat{A}$ / CBD	与 stable Encrypt: <code>F203_AHAT16_BLOCK_DIM=2</code> ; 双 AIV 分片 <code>SampleNTT</code> ; <code>PRF(r, \cdot) + \text{CBD}_{\eta=2}</code> 产 $y \ e_1 \ e_2$ （ r 由设备 G 写出, 非 Host 输入）
Compute+pack	与 stable Encrypt: SIM 内联 <code>f203_encrypt_118_119</code> ; NTT S1-S3 禁 Gather ; <code>Compress_{11/5} + ByteEncode_{11/5}</code>
Host	SIM 2 / CPU 5; <code>ek_kem+m+LUT</code> $\rightarrow$ 仅 $c+K$
宏默认	与 Encrypt 生产全量一致; 禁止 关 pack / 关向量路 径作默认验收

禁止 将「Host 注入 r 」「关 `ByteEncode`」「分段只验 Encrypt」等作为默认 `run.sh` 验收; 调试对照须显式覆盖 `env`, 并在头注释标「非默认」。

1.5 目录布局（计划; 写码阶段落地）

```

stable-fips203-mlkem-kem-encaps-k4/
  stable-fips203-mlkem-kem-encaps-k4-实现方案-customspec.tex/.pdf
  main_kem_encaps.cpp           # Host: SIM 2 / CPU 5; D2H c+K
  f203_kem_enc_layout.h         # I/O 常量 (1568/32)
  kem/
    f203_kem_enc_init.hpp       # KemEncInitHead (H/G)
    f203_kem_enc_prep_entry.cpp # 注册 f203_kem_enc_prep
  prep/                          # vendored Alg.14 行 3-15
  compute/                      # vendored 行 2/16-22 + 内联 pack
  cmake/encaps/CMakeLists.txt
  scripts/                     # gen_data / verify / golden 胶水
  run.sh
  SELF_CONTAINED.md           # 写码阶段补
  STATUS.md                   # 写码验收后补

```

vendor 来源（一次性, 写码时复制后切断）: `examples/stable/stable-fips203-mlkem-pke-encrypt-k4/` 的 `prep/`、`compute/`、Encrypt layout 头与必要 Host 胶水; KEM 增量对照（只读行为, 禁止编译依赖）`ascendc-tests/pass-fix-f203-alg20-kem-encaps-device-k4/kem/`。

2 数学语义 ($Alg.20 \rightarrow Alg.17 \rightarrow Alg.14$)

2.1 参数集

ml_kem_1024: $n = 256$, $q = 3329$, $k = 4$; $d_u = 11$, $d_v = 5$; $\eta = 2$ 。(与本仓 PKE Encrypt / KEM KeyGen 交付同档; 勿与 ML-KEM-768 混用。)

2.2 Algorithm 20 (对外业务) 与 Alg.17 (设备 I/O)

对外 API 可对应 Alg.20 (随机采样 m 再调 internal)。本用例设备核锁定 **Alg.17 形态** (与 device 探针一致):

$ek \in \mathbb{B}^{1568}$, $m \in \mathbb{B}^{32}$	Host H2D: ek_kem , m
$h \leftarrow H(ek) = \text{SHA3-256}(ek)$	Launch-1 前段 UB
$(K r) \leftarrow G(m h) = \text{SHA3-512}(m h)$	$K, r \in \mathbb{B}^{32}$
$c \leftarrow \text{K-PKE.Encrypt}(ek, m, r)$	Alg.14: 随机性输入即为 r
输出 (K, c)	

- m : **GM 输入** (input/m.bin)。熵来源 (Host CSPRNG / 定点文件) 属 harness, **不改变核契约**。
- r : **仅** device 写 workspace GM; prep 以 r 为种子做 PRF+CBD (得 $y||e_1||e_2$); **不落盘**为交付物。
- h : UB only, **不落盘**。

2.3 相对 PKE Encrypt 的增量

项	PKE Encrypt (Alg.14)	本 KEM Encaps
生产输入	$ek+m+r$ (Alg.14)	$ek+m$ (r 非 Host 输入)
$r \in \mathbb{B}^{32}$	Host 预填	设备 G 后半写出
共享秘密 K	无	G 前半 $\rightarrow$ K.bin
密文 c	1568 B	同 Encrypt
Launch 数	SIM 2 / CPU 5	同 (头并入 L1)

2.4 liboqs 对拍口径

验收对齐 QQS_KEM_ml_kem_1024 的 encaps_derand (固定 ek 、给定 m):

```
input:  ek_kem[1568], m[32]
output: c[1568], K[32]      # 逐字节 == liboqs
```

Golden / KAT 仅验 I/O 等价; **禁止**以「与 device / correctness 源码同构」验收。

2.5 Alg.14 主体 (设备, 与 Encrypt 同)

行 2-22 与 exp-fips203-mlkem-pke-encrypt-k4-实现方案-customspec.tex 「数学语义」节同构 (本文不重复展开公式)。本阶段**不做**: Decaps (Alg.21)、KeyGen、为头段单独开核。

3 编排与 Launch

3.1 SIM (生产路径)

aclInit / CreateStream

```
Launch-1: f203_kem_enc_prep          // AIV_ONLY, blockDim=2
          block0: KemEncInitHead(ek, m → K_gm, r_gm)    // H/G
          dual AIV: BuildEncryptPrepSinglePipe           // A-hat + CBD(y||e1||e2) <- r
Launch-2: f203_encrypt_l18_l19       // MIX_AIC_1_2, blockDim=1
          m_gm + A-hat + (y||e1||e2) -> c (内联 pack)
aclrtSynchronizeStream               // 仅 L2 后一次
D2H: c.bin (1568), K.bin (32)
aclFinalize
```

3.2 CPU (tikicpu 分叉)

沿用 stable Encrypt **5 launch**: #1 换为 f203_kem_enc_prep (头 + $\hat{A}+y||e_1||e_2$); #2-#5 同 Encrypt (NTT / 内积 / INTT / pack)。CPU 路径**不是**与 SIM 同构的完整设备 Encrypt; 验收仍以 $c+K \max=0$ 为准。须用 ASCENDC_BUILD_CPU/ASCENDC_BUILD_SIM + ASCENDC_SIM_HOST_MODE, 禁止 per-probe 宏分叉。

3.3 相对 correctness (vendor G5)

项	correctness-k4	本方案 / device
Encrypt	frozen G5 vendor	stable 对齐 / vendored
KEM 头	常与 oracle 拼装	并入 prep; 设备 SHA3
r	可能 Host/oracle	仅 device G
SIM tick	历史 $\approx 1.0M$ (旧)	目标 $\approx$ device 721k

4 AI Core 规模

段	核类型 / blockDim / 角色
Prep (L1)	KERNEL_TYPE_AIV_ONLY; blockDim=2; 双 AIV 各 8 poly 分片 $\hat{A}$; 仅 block0 执行 KemEncInitHead 与 PRF+CBD
Compute (L2)	KERNEL_TYPE_MIX_AIC_1_2; blockDim=1; 1 AIC + 2 AIV; NTT/INTT/内积/内联 pack

KEM 头

不新增 AI Core、不增 blockDim、不增 launch

内部命名: prep aiv=2; compute aicore=1 (MIX 1:2)。**选型理由:** 复用已验收 Encrypt 拓扑; KEM 增量仅为 prep 前段标量 SHA3 ($\leq 1568\text{B}$ 摘要 + 64B G), 不值得单独 AIV_ONLY launch (device 已 PASS)。**禁止实现擅自改 blockDim 却不回写本规格。**

串行占用: prep 2 Vector AI Core; compute 1 颗 AI Core。CPU [SUCCESS] [AIC_*] 为 tikicpu artifact; 以 SIM profile_*.toml / Total tick 为准。

5 GM 布局与头段同步

GM 缓冲	尺寸	用途
ek_kem_gm	1568	H2D; 头读 + Encrypt 读 ρ /decode
m_gm	32	H2D; 头读 + Launch-2 仍用 (μ)
K_gm	32	头写; = G 前半; 唯一新增 交付 GM
r_gm	32	头写; = r ; prep 以 r 为种子做 PRF+CBD; 不 D2H
$\hat{A} / (y\ e_1\ e_2)$	同 Encrypt	L1 写 / L2 读; 禁落盘
ws+LUT	同 Encrypt	L2
c_gm	1568	L2 写; D2H

不新增: h_gm、Host 侧 r 输入文件契约。

5.1 KemEncInitHead (仅 block0)

1. 自 m_gm 读 $m[32]$ UB。
2. 自 ek_gm 读 1568B UB $\rightarrow$ Sha3OneShot (mdlen=32) $\rightarrow h$ 。
3. 拼 $m\|h$ (64B) $\rightarrow$ Sha3OneShot (mdlen=64) $\rightarrow K\|r$ 。
4. 写 K_gm、 $r \rightarrow r_gm$ (标量逐字节或小块 DataCopy)。

5.2 双 AIV 与头段次序 (强制)

- SIM/NPU:GetBlockIdx()==0 先完成 KemEncInitHead, 再与 block1 并行跑 BuildEncryptPrepSinglePip
SampleNTT 只依赖 ek 尾 ρ , 不依赖 r ; CBD/PRF **仅** block0, 故须保证 block0 在 CBD 前
已写完 r_gm。
- CPU (ASCENDC_CPU_DEBUG 且 blockDim=2): 单入口串行两次 $\hat{A}$ 分片; 头**只跑一次** (对齐
device 入口)。
- 不要求 KeyGen 式 Encode 后 SyncAll (本用例无双 AIV 分片写再由他核读完整 sk 的场景);
但本核写 r 后、CBD 读前须 PipeBarrier (若写路径与向量路径混用)。

6 踩坑与强制条款 (换 Agent 必读)

条款	要求
禁 Host 伪头	禁止 Host SHA3 写 K/r 再 launch Encrypt 冒充 Encaps
禁预填 r	生产 <code>gen_data</code> 不得提供 Host 侧 r 输入契约
禁 G5 / frozen	禁止 <code>vendor_sync</code> frozen G5; 禁抄 frozen 源码
禁第 3 launch	H/G 必须并入 L1; 否决 correctness 式独立 kem 头核
禁编译依赖探针	device 只读行为; incubating 须 vendored 自包含
已晋级 stable	2026-07-15 # 验收 # 自 incubating 复制; 预研副本保留
run.sh 坑	不得在 <code>source env.sh/setenv</code> 前 <code>set -e</code> (<code>grep -q</code> 非零会静默退出; 见 qa 2026-07-15)
cwd	二进制须在用例根 cwd 跑 (相对路径 <code>./input/...</code>)

7 全量 API 清单 (查阅 CANN 9.0.0)

说明: Encrypt 主体 (prep / NTT / MMAD / Compress / ByteEncode) API 与 `exp-fips203-mlkem-pke-encrypt-k4` / stable Encrypt 同集, 已在 `library/documents/CANN-AscendC` 算子开发接口参考-查阅索引.md 有记录; 下表列本用例须显式锁定的主路径 API (含 KEM 头)。分类按 PDF 第 2 章; 在线 ID 为社区 `atlasascendc_api_07_*`。

API	分类	在线 ID	A2	输入/输出 (本用例)	备注
GlobalTensor	2.2	07_0007	✓	GM 描述	SetGlobalBuffer
LocalTensor	2.2	07_0006	✓	UB 描述	Encrypt 向量段
GetBlockIdx	2.3 系统	07_0185	✓	核索引	头仅 block0; prep 分片
GetSubBlockIdx	2.3 系统	07_0186 邻	✓	AIC/AIV 子块	L2 MIX
KERNEL_TYPE_AIV_ONLY	Utils	编程指南	✓	L1	Prep+KEM 头
KERNEL_TYPE_MIX_AIC_1_2	Utils	编程指南	✓	L2	1 AIC : 2 AIV
TPipe/TQueue/TBuf	2.3 资源	07_0108/0137	✓	管道	Prep/Compute
DataCopy	2.3.1	07_0101	✓	uint8/int32 GM↔UB	头段可选; Encrypt 主路径
DataCopyPad	2.3.1	07_0265	✓	可选跨步	头主路径 不用
PipeBarrier	同步	常用	✓	本核 MTE/V	头写 r 后可见性 (若需)
CrossCoreSetFlag	同步	API 列表	✓	AIC↔AIV	L2 FSM

API	分类	在线 ID	A2	输入/输出 (本用例)	备注
CrossCoreWaitFlag	同步	API 列表	✓	同上	与 Encrypt 同构
ShiftRight/Muls/Adds/Add/Sub	2.3.3	07_0059/55/54/35/36	int32		Encrypt 向量链
Duplicate	2.3.3	07_0041 邻	✓	填充	Prep/对齐
GatherMask	2.3.3.4	07_0071	✓	掩码收集	rej compact (非 NTT S1-S3)
Gather	2.3.3	07_0071 邻	✓	—	NTT S1-S3 禁用 ; Encode 后另议
Cube MMAD / AicMmad	2.3.2	教程 + 封装	✓	int8×int8→int32	Stage2

7.1 设备哈希 (非 CANN 矢量 API; 工程共享库)

符号	契约
F203SeDeviceKeccak::Sha3OneShot	library/shared/keccak_f1600_kernel/fips203_device_sha3.hpp; __aicore__ one-shot; mdlen=32 ($H(ek)$) / 64 ($G(m h)$)。日后可换第三方 AscendC SHA3, 勿改 G 输入拼接顺序 ($m h$) 与 $K r$ 切分。

7.2 评估过但未采用

方案	结论
独立 launch 仅算 H/G	多 sync; device 已否决; 禁止
Host 预填 r + 纯 Encrypt	非 Alg.17 设备 Encaps; 禁止 作生产
Host SHA3 写 K 再只跑 Encrypt	伪设备; 禁止
frozen / correctness G5 Encrypt	路线关闭; 禁止 抄码
编译期长期 STABLE_ENCRYPT_ROOT	incubating 须自包含 vendor; 探针例外
NTT S1-S3 内 Gather	Tag5T 禁令; 沿用 Encrypt

未绿即复制 stable 禁止（对齐 KeyGen 教训）

Encaps

8 数学步骤 → API 覆盖率

数学步骤	主路径	覆盖	缺口/备注
$H(ek)$	SHA3-256 共享库	标量	1568 B 入 UB + one-shot
$G(m\ h) \rightarrow K\ r$	SHA3-512 共享库	标量	写 K_{gm}/r_{gm}
SampleNTT $\hat{A}$	SHAKE + rej 向量	部分	与 stable Encrypt Prep 同
PRF+CBD $_{\eta=2}$	向量/标量混合	高	r 种子改自设备 G
NTT / 内积 / INTT	MIX 向量 +Cube	高	S1-S3 无 Gather
Compress+Encode→ c	向量 Compress + 标 量 pack	高	同 Encrypt 尾
写出 K	标量/DataCopy	100% 搬运	32 B

主路径结论：Encrypt 计算段与 stable 同覆盖率；KEM 增量哈希为**显式标量缺口**（设备共享 Keccak，非矢量 API）。 K/r 落点为小块搬运。

9 验证

9.1 Golden 角色

Host / liboqs 仅提供合法输入与期望输出（黑盒 oracle）。**禁止**以「与 device / correctness 源码同构」为验收；仅 **I/O 等价**。

9.2 生产验收（默认；写码完成后）

```
cd examples/incubating/exp-fips203-mlkem-kem-encaps-k4
bash run.sh -r cpu -v Ascend910B4
bash run.sh -r sim -v Ascend910B4 # 默认全量；run.sh 内自动 SIM_DIRECT=1
# 检查 output/c.bin (1568B) + K.bin (32B)；max=0
```

对照 oracle(写码阶段):同 ek/m 下与 pass-fix-f203-alg20-kem-encaps-device-k4 及/或 liboqs encaps_derand 的 c/K 逐字节一致。仓库脚本(写码接线后):`bash scripts/liboqs_kem_encaps_batch.sh` (ENCAPS_DIR 可覆写为本目录)。

禁止把一长串与 default 相同的 `HAT_*=0/1` 写入标准验收命令。

9.3 Gate 建议 (写码阶段, 非本规格交付物)

Gate	验收
G0	CMake+run.sh 壳; kernel 结束
G1	固定 ek/m: 头后 r/K 可对照 host tiny_sha3 (调试)
G2	全链 c/K vs liboqs encaps_derand; CPU max=0
G3	SIM 同 G2; 更新 tick; 根目录无 stray dump
G4	默认 run.sh CPU+SIM PASS; 建议 CPU 多轮清零 output/
G5	分项 KAT: 建议 CPU×10 + SIM×3 (对齐 device)

10 性能 (SIM 910B4 参考)

- stable PKE Encrypt: Total tick $\approx$ **627590** (全链)。
- device KEM Encaps (本基线): Total tick **721010** (头 SHA3 + Encrypt)。
- 本 exp 写码验收目标: 与 device **同量级**; **远超** (无日志挂死) 视为回归。
- $\sim 15\text{s}$ 仅为 Tag5T NTT 全流程性能定标, 不套用本用例防挂死预算; 预算写进本用例 run.sh (建议默认 $\geq 600\text{s}$, 与 Encrypt/device 同量级)。

11 写码衔接 (本规格确认后)

1. 用户确认本 customspec (含 §6、I/O、Launch、自包含)。
2. 用户明确 **【预研】** / 「可以写代码」并指明本路径后, 按 .cursor/skills/pre-research/S KILL.md 在本目录落地 vendored 源码 (当前目录无实现可抄; device 仅对照行为)。
3. Golden: liboqs / device 对照; 登记表若需 Encaps 专表则另开 docs/specs/ (缺项须停下请用户补来源)。
4. incubating 经验收阶梯稳定后, 用户 # 交付 # 再复制晋级 examples/stable/stable-fips203-mlkem-kem-encaps-k4/ (名称待交付时确认); **禁止**未绿先晋级。